T5A2

The data are $f_i = f(x_i)$ for $i = 0, 1, \ldots, n$. I approximate them with the polynomial

$$p_m(x) = \sum_{k=0}^{m} c_k x^k,$$

and I look for the coefficients $c_0, c_1, \ldots, c_m$ that make

$$\chi^2 = \sum_{i=0}^{n} [p_m(x_i) - f_i]^2$$

as small as possible.

1) Analytical formulas for the coefficients

At the minimum of χ^2, the derivative to every coefficient is zero:

$$\frac{\partial \chi^2}{\partial c_l} = 0, \qquad l = 0, 1, \ldots, m.$$

In $p_m(x_i) = c_0 + c_1 x_i + \cdots + c_m x_i^m$ only the term $c_l x_i^l$ depends on c_l, so $\partial p_m(x_i)/\partial c_l = x_i^l$. With the chain rule,

$$\frac{\partial \chi^2}{\partial c_l} = \sum_{i=0}^{n} 2[p_m(x_i) - f_i] x_i^l = 0.$$

I divide by 2 and put in $p_m(x_i) = \sum_k c_k x_i^k$:

$$\sum_{i=0}^{n} \sum_{k=0}^{m} c_k x_i^k x_i^l = \sum_{i=0}^{n} f_i x_i^l.$$

I swap the two sums, and I take c_k out of the sum over i because it does not depend on i:

$$\sum_{k=0}^{m} \left(\sum_{i=0}^{n} x_i^{l+k} \right) c_k = \sum_{i=0}^{n} f_i x_i^l, \qquad l = 0, 1, \ldots, m.$$

These sums only depend on the data, so I call them

$$s_j = \sum_{i=0}^{n} x_i^j \quad (j = 0, 1, \ldots, 2m), \qquad t_l = \sum_{i=0}^{n} f_i x_i^l \quad (l = 0, 1, \ldots, m).$$

I need s_j up to $j = 2m$, because $l + k$ goes up to $2m$. Note that $s_0 = n + 1$, because $x_i^0 = 1$. Now the equations are

$$\sum_{k=0}^{m} s_{l+k} c_k = t_l, \qquad l = 0, 1, \ldots, m.$$

These are $m+1$ linear equations for the $m+1$ unknowns c_k. In matrix form, $Sc = t$:

$$\begin{pmatrix} s_0 & s_1 & \cdots & s_m \\ s_1 & s_2 & \cdots & s_{m+1} \\ \vdots & \vdots & \ddots & \vdots \\ s_m & s_{m+1} & \cdots & s_{2m} \end{pmatrix} \begin{pmatrix} c_0 \\ c_1 \\ \vdots \\ c_m \end{pmatrix} = \begin{pmatrix} t_0 \\ t_1 \\ \vdots \\ t_m \end{pmatrix},$$

so $S_{lk} = s_{l+k}$. If $\det S \neq 0$, Cramer's rule gives every coefficient:

$$c_k = \frac{\det S^{(k)}}{\det S}, \qquad k = 0, 1, \ldots, m,$$

where $S^{(k)}$ is S with column k replaced by t.

Now I check that $\det S \neq 0$ when at least $m+1$ of the x_i are different (so $n \geq m$). I take any vector $v = (v_0, v_1, \ldots, v_m) \neq 0$ and the polynomial $q(x) = \sum_{k=0}^{m} v_k x^k$. With $s_{l+k} = \sum_i x_i^l x_i^k$,

$$v^T S v = \sum_{l=0}^{m} \sum_{k=0}^{m} v_l s_{l+k} v_k = \sum_{i=0}^{n} \left(\sum_{l=0}^{m} v_l x_i^l \right) \left(\sum_{k=0}^{m} v_k x_i^k \right) = \sum_{i=0}^{n} q(x_i)^2.$$

q is not the zero polynomial and its degree is at most m, so it has at most m roots. So $q(x_i) \neq 0$ for at least one of the $m+1$ different x_i, and $v^T S v > 0$. If $\det S = 0$, there would be a $v \neq 0$ with $Sv = 0$, and then $v^T S v = 0$. That is not possible, so $\det S \neq 0$.

I also check that this solution is a minimum of χ^2. I change the coefficients to $c_k + \delta_k$ with $\delta \neq 0$. Then $p_m(x_i)$ changes by $q(x_i) = \sum_k \delta_k x_i^k$, and with $(a+b)^2 = a^2 + 2ab + b^2$:

$$\begin{aligned} \chi^2(c+\delta) &= \sum_{i=0}^{n} [p_m(x_i) - f_i + q(x_i)]^2 \\ &= \sum_{i=0}^{n} [p_m(x_i) - f_i]^2 + \sum_{l=0}^{m} \delta_l \left(\sum_{i=0}^{n} 2[p_m(x_i) - f_i] x_i^l \right) + \sum_{i=0}^{n} q(x_i)^2 \\ &= \chi^2(c) + \sum_{l=0}^{m} \delta_l \frac{\partial \chi^2}{\partial c_l} + \delta^T S \delta. \end{aligned}$$

The middle term is zero, because every $\partial\chi^2/\partial c_l = 0$ at the solution. The last term is $\delta^T S \delta > 0$, the same as $v^T S v$ above. So $\chi^2(c+\delta) > \chi^2(c)$ for every $\delta \neq 0$, and the solution is the minimum.

2) Main steps of the numerical algorithm

Numerically I do not use Cramer's rule. It needs $m+2$ determinants, and one determinant costs about as much as one Gaussian elimination. Solving $Sc = t$ directly needs only one elimination.

Step 1: Input the data x_i, f_i for $i = 0, 1, \ldots, n$ and the order m. At least $m+1$ of the x_i must be different, so that $\det S \neq 0$ (see part 1).

Step 2: Compute the sums s_j for $j = 0, 1, \ldots, 2m$ and t_l for $l = 0, 1, \ldots, m$ in one loop over the data points. I get the powers by multiplying with x_i every time, $x_i^{j+1} = x_i^j \cdot x_i$.

Step 3: Fill the $(m+1) \times (m+1)$ matrix S with $S_{lk} = s_{l+k}$, and the vector t.

Step 4: Solve $Sc = t$ with Gaussian elimination with partial pivoting (section 5.3 in the book). For each column j, I swap row j with the row (from j down) that has the largest $|S_{rj}|$, and I swap the

same two elements of t. This way I never divide by a zero or very small pivot. Then I subtract S_{rj}/S_{jj} times row j from every row r below it (also in t), so everything below the diagonal becomes zero. After that I find the coefficients with back substitution, from c_m to c_0:

$$c_l = \frac{1}{S_{ll}} \left(t_l - \sum_{k=l+1}^{m} S_{lk} c_k \right),$$

with S and t as they are after the elimination.

Step 5: Output $c_0, c_1, \ldots, c_m$.

3) Subprogram

The subprogram for steps 1 to 5, and the Gaussian elimination it uses:

```
subprogram leastSquares(x[0..n], f[0..n], m) returns c[0..m]
    # Step 1: the input is the data x_i, f_i and the order m
    # Step 2: the sums s_j and t_l
    s[0..2m] = 0
    t[0..m] = 0
    for i = 0 to n #loop over the data points
        xp = 1 #xp = x_i^j, starts with x_i^0 = 1
        for j = 0 to 2m
            s[j] = s[j] + xp
            if j <= m: t[j] = t[j] + f[i]*xp #t only goes up to m
            xp = xp*x[i] #next power of x_i
    # Step 3: fill the matrix S with S_lk = s_(l+k)
    for l = 0 to m
        for k = 0 to m
            S[l][k] = s[l+k]
    # Step 4: solve S c = t
    c = gauss(S, t)
    # Step 5: output the coefficients
    return c

subprogram gauss(S[0..m][0..m], t[0..m]) returns c[0..m]
    # elimination with partial pivoting, column by column
    for j = 0 to m-1
        # largest pivot, so I never divide by zero or a very small number
        piv = the row r >= j with the largest |S[r][j]|
        swap rows j and piv of S, and t[j] and t[piv]
        for r = j+1 to m
            fac = S[r][j]/S[j][j] #chosen so that S[r][j] becomes 0
            for k = j to m
                S[r][k] = S[r][k] - fac*S[j][k]
            t[r] = t[r] - fac*t[j] #same row operation on t
    # S is upper triangular now, so back substitution from the last row up
    for l = m down to 0
        c[l] = (t[l] - sum of S[l][k]*c[k] for k = l+1 to m)/S[l][l]
    return c
```